

CS456 Design and Analysis of Algorithms

Project 4 - Network Flow

Taxi Dispatch as a Minimum Cost Maximum Flow Problem: A Report on Network Flow Optimization

Deepshan Adhikari (800846035)
deepadh@siue.edu

1 Introduction

Ride-hailing services such as Uber, Lyft, Didi, Grab, and Kakao T handled millions of ride-hailing orders per day in 2019 [1]. A major challenge for these platforms is maintaining a proper number of vehicles in each service area. If there are not enough vehicles located nearby when a customer requests a ride, the service provider is unable to serve the request. This leads to bad user experience and potential revenue loss [1].

This is a serious problem because ride-hailing has become a major part of urban transportation. Millions of people depend on these services every day. If a platform cannot serve requests efficiently, customers will switch to competitors. Drivers will also leave if they cannot earn enough income. Traditional taxi systems also suffer from inefficiencies due to uncoordinated actions as system capacity and customer demand change [2].

Network flow is an appropriate tool for solving this problem because taxi dispatch involves moving discrete vehicles through a network of locations over time. The number of vehicles that can move between areas is limited by how many are available. The number of requests that can be served is limited by how many vehicles are present. These features map naturally to capacities and flows in a directed graph [6]. The problem can be converted into a Minimum Cost Maximum Flow (MCMF) problem. The MCMF algorithm finds a flow that maximizes the number of served requests while minimizing the total repositioning cost [1]. The max-flow min-cut theorem guarantees that the maximum flow value equals the minimum cut capacity [3, 4]. The minimum-cost maximum-flow algorithm is responsible for finding the least-cost flow among all maximum flows [5].

2 Network Flow Model

2.1 Representing the Taxi-Dispatch Problem

A flow network is a directed graph $G = (V, E)$, where V is a set of vertices and E is a set of directed edges. Each edge (u, v) has a capacity $c(u, v)$. A source s supplies flow, and a sink t receives it. The objective is to maximize the total flow from source to sink without exceeding capacities or violating flow conservation [3, 4].

For taxi dispatch, the source represents available vehicle supply. Intermediate vertices represent service areas. The sink represents ride requests that can be served. Edge capacities limit the number of vehicles or assignments. Flow value represents the number of ride requests served in the modeled time period [1].

2.2 The Model Used in the Research Paper

Kim et al. [1] divide the service region into regular hexagonal cells. Each cell represents the area a vehicle can cover within a unit time interval. One day is divided into 144 time steps of 10 minutes each. Their single-step network uses two sets of vertices: V_i (vehicle-side state of cell i) and W_j (request-side state of cell j). The main components are: (1) an edge from source s to each V_i with capacity $d_{i,0}$, representing available vehicles; (2) edges from V_i to W_j with infinite capacity and cost $c_{i \rightarrow j,0}$; and (3) an edge from each W_j to sink t with capacity $r_{j,0}$, representing ride requests [1]. The graph represents a single dispatch step. The predictive approach repeats or reoptimizes decisions over a horizon of multiple time steps.

2.3 Notation

Symbol	Quantity
N_i	i -th hexagonal cell (node)
$d_{i,t}$	number of drivers in N_i in time step t
$r_{i,t}$	number of ride requests in N_i in time step t
$x_{i \rightarrow j,t}$	number of vehicles moving from N_i to N_j in time step t
$c_{i \rightarrow j,t}$	cost of relocating from N_i to N_j in time step t

Table 1: Notation used in the taxi dispatch model

2.4 Multi-Objective Taxi Dispatch Problem

The goal of the fleet controller is twofold: to maximize the number of served requests and to minimize the total cost of relocating vehicles. This is a multi-objective optimization problem where objectives often conflict, and the solution set forms a Pareto frontier. The number of served requests in cell N_i in time step t is:

$$g_{i,t}(x_t) = \min \left\{ d_{i,t} + \sum_{j=1}^n x_{j \rightarrow i,t} - \sum_{k=1}^n x_{i \rightarrow k,t}, r_{i,t} \right\} \quad (1)$$

The first objective is

$$f_+(x) = \sum_{t=0}^{T-1} \sum_{i=1}^n g_{i,t}(x_t).$$

The cost of repositioning vehicles is

$$h_t(x_t) = \sum_{i,j=1}^n c_{i \rightarrow j,t} x_{i \rightarrow j,t}.$$

The second objective is

$$f_-(x) = \sum_{t=0}^{T-1} h_t(x_t).$$

The constraints are

$$d_{i,t+1} = d_{i,t} + \sum_{j=1}^n x_{j \rightarrow i,t} - \sum_{k=1}^n x_{i \rightarrow k,t} - g_{i,t}(x_t),$$

$$\sum_{j=1}^n x_{i \rightarrow j,t} \leq d_{i,t}, \quad x_{i \rightarrow j,t} \geq 0.$$

The problem is

$$\max (f_+(x), -f_-(x))$$

The MCMF formulation finds a flow that maximizes the number of served requests first, then minimizes the repositioning cost among all maximum flows. This corresponds to choosing a specific point on the Pareto frontier where the number of served requests is maximized and the cost is minimized.

2.5 Converting to a Network Flow Problem

For each cell i , two copies are created: V_i and W_i . The movement of vehicles is represented by flow from V_i to W_j . The flow capacity between V_i and W_j is infinite if cells i and j are the same or adjacent, and zero otherwise [1]. An artificial source S and sink T are introduced. The flow capacity from S to V_i equals $d_{i,0}$. The flow capacity from W_j to T equals $r_{j,0}$. The cost of moving from V_i to W_j is $c_{i \rightarrow j,0}$. This construction follows the standard technique for converting assignment problems into flow networks [3]. The minimum cost maximum flow problem seeks a flow of maximum value with minimum total cost [5].

Taxi Dispatch	Network Flow Problem
$d_{i,t}$ (number of drivers)	flow capacity from S to V_i
$r_{i,t}$ (number of requests)	flow capacity from W_i to T
$c_{i \rightarrow j,t}$ (reposition costs)	flow costs

Table 2: Relationship between taxi dispatch and network flow

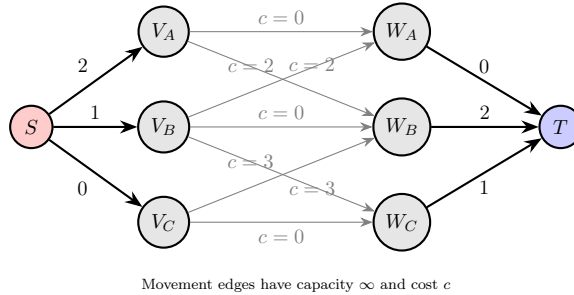


Figure 1: Network diagram for the worked example with 3 cells. Source edges carry driver counts, movement edges have infinite capacity with repositioning costs, and request edges carry request counts.

The maximum flow value equals the maximum number of ride requests that can be served [1]. The MCMF algorithm repeatedly finds minimum-cost augmenting paths in the residual graph [5]. An augmenting path is a path from source to sink where every edge has positive residual capacity [3]. The max-flow min-cut theorem establishes that the maximum flow value equals the minimum cut capacity. The minimum-cost maximum-flow algorithm finds the least-cost flow among all maximum flows [5].

3 Worked Example

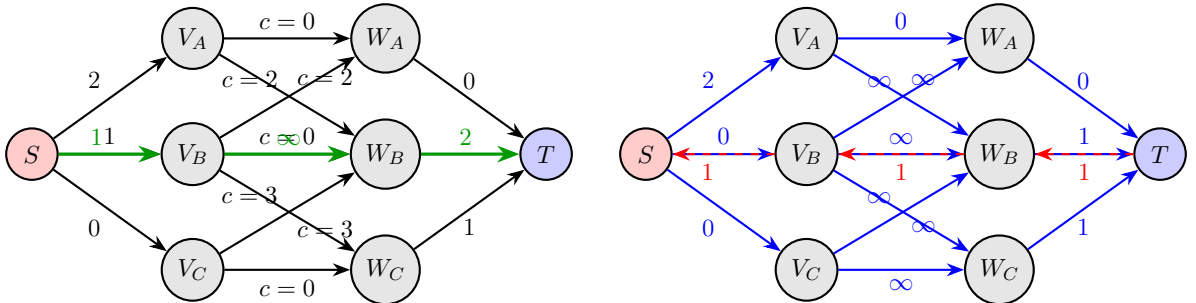
A small example is constructed with three cells A , B , and C arranged in a line. Cell A is adjacent to B , and cell B is adjacent to both A and C . One time step is considered. Cell A has 2 drivers and 0 requests; Cell B has 1 driver and 2 requests; Cell C has 0 drivers and 1 request. The movement costs are: $V_A \rightarrow W_A$ cost 0; $V_A \rightarrow W_B$ cost 2; $V_B \rightarrow W_A$ cost 2; $V_B \rightarrow W_B$ cost 0; $V_B \rightarrow W_C$ cost 3; $V_C \rightarrow W_B$ cost 3; $V_C \rightarrow W_C$ cost 0. All other movements are not allowed.

The network has source S , sink T , and nodes $V_A, V_B, V_C, W_A, W_B, W_C$. Source edges are $S \rightarrow V_A$ (capacity 2), $S \rightarrow V_B$ (capacity 1), and $S \rightarrow V_C$ (capacity 0). Request edges are $W_A \rightarrow T$ (capacity 0), $W_B \rightarrow T$ (capacity 2), and $W_C \rightarrow T$ (capacity 1). Movement edges are shown in Figure 1.

3.1 Augmenting Path and Residual Graph

Figure 2 shows the augmenting path and the resulting residual graph side by side. The left figure highlights the augmenting path $S \rightarrow V_B \rightarrow W_B \rightarrow T$ in green. This is the minimum-cost path because its cost is 0. The residual capacities along this path are 1, ∞ , and 2. The bottleneck is $\min(1, \infty, 2) = 1$. The cost of this path is $0 + 0 + 0 = 0$. Sending 1 unit of flow along this path increases the flow value by 1.

The right figure shows the residual graph after this augmentation. Forward edges are shown in blue. Reverse edges are shown in red dashed. The forward edge $S \rightarrow V_B$ has residual capacity 0 because it is saturated. The forward edge $W_B \rightarrow T$ has residual capacity 1. Reverse edges $V_B \rightarrow S$, $W_B \rightarrow V_B$, and $T \rightarrow W_B$ have residual capacity 1 because 1 unit of flow was sent along the original edges. These reverse edges allow the algorithm to cancel flow if a better solution is found later.



(a) Augmenting path highlighted in green. (b) Residual graph after augmentation.

Figure 2: Augmenting path and resulting residual graph. Blue solid: forward residual capacity. Red dashed: reverse residual capacity.

3.2 Final Maximum Flow

The algorithm continues until no augmenting paths remain. The final assignment serves all three requests: both vehicles from cell A move to cell B , and the vehicle from cell B moves to cell C . The total flow is 3, which is the maximum possible because there are only three requests. The total repositioning cost is

$$2 \times 2 + 1 \times 3 = 7.$$

The assignment is also minimum-cost among the flows that serve all three requests. The request in cell C can only be served by the vehicle from cell B , which costs 3. The two requests in cell B are then served by the two vehicles from cell A , at a total cost of $2 \times 2 = 4$. Therefore, the minimum total cost for serving all three requests is 7. Thus, the solution is a maximum flow of 3 with a minimum total cost of 7.

4 Discussion and Limitations

Network flow works well for this problem because the transportation system has a natural graph structure. Vehicles are discrete entities that move between hexagonal cells. The number of vehicles that can move between cells is limited by availability. The number of requests that can be served is limited by the number of vehicles present. These features map naturally to capacities and flows in a directed graph. The max-flow min-cut theorem provides a strong theoretical foundation [3, 4].

The MCMF algorithm provides a globally optimal solution to the formulated network-flow problem [5]. Because the network has integer capacities, an optimal flow can be chosen to have integer values. This produces physically meaningful vehicle assignments without requiring rounding. The time-greedy property provides a worst-case performance guarantee regardless of prediction accuracy [1]. The model assumes that all taxis are homogeneous and always follow commands without exception. The controller has perfect knowledge of current driver and request distributions. Current observed vehicle and request information is known, but future demand predictions are uncertain [1].

The model does not account for real-time traffic variations. Driver behavior such as refusing assignments is not modeled. The discretization of the city into hexagons may introduce errors for short trips. The model may fail when special events cause sudden demand spikes that were not predicted. It may also fail when many drivers go offline unexpectedly, creating supply shortages. The paper reports performance degradation on Days 3, 4, and 13 due to special events held in Seoul on those days [1].

The same network flow framework can be applied to rebalancing bike-sharing systems, ambulance relocation and dispatching, vehicle routing with time windows, and logistics fleet management.

5 Research Paper Analysis

5.1 Paper Objective

The paper by Kim et al. [1] addresses the large-scale fleet management problem in ride-hailing systems. The authors aim to develop a predictive taxi dispatch method that serves as many ride requests as possible while minimizing the cost of relocating idle vehicles. The method must handle the uncertainty of future demand and provide real-time dispatch decisions. The paper proposes a receding horizon optimization approach that converts the multi-objective taxi dispatch problem into a network flow problem.

5.2 How Network Flow Was Used

The authors convert the multi-objective receding horizon optimization problem into a Minimum Cost Maximum Flow (MCMF) problem [1]. The urban transportation system is modeled as a flow network with regular hexagonal cells as nodes. The source S connects to vehicle-side nodes V_i with capacity $d_{i,0}$. Movement edges connect V_i to request-side nodes W_j with infinite capacity and cost $c_{i \rightarrow j,0}$. Request-side nodes W_j connect to the sink T with capacity $r_{j,0}$. The MCMF algorithm finds a flow that maximizes the number of served requests while minimizing the total repositioning cost [1].

5.3 Why Network Flow Was Selected

The authors selected network flow for three main reasons [1]. First, the transportation system has an inherent graph structure. Vehicles move between discrete locations, and requests occur at specific locations. Second, the MCMF algorithm provides a globally optimal solution to the formulated network-flow problem. Third, integer capacities produce integer-valued solutions, which is physically meaningful because the decision variable represents the number of relocating vehicles.

5.4 Advantages

The proposed approach has four advantages [1]. The integer property ensures that the optimal flow plan uses integer-valued vehicle assignments, so no rounding is required. The time-greedy property maximizes the number of served requests in the current time step even when future demand is uncertain. In the reported experiments, the LP method was approximately 880 times faster than the classical MCMF algorithm when the prediction horizon was 16. The method also achieved 97.16% of the performance of an oracle with perfect future knowledge, based on the relative profit metric.

5.5 Disadvantages

The proposed approach has four limitations [1]. The model assumes that all taxis are homogeneous and always follow commands without exception, which may not hold in practice. The model relies on demand predictions, but these predictions can be inaccurate, especially during special events. The model does not consider real-time traffic conditions, which can affect the ability of vehicles to move between cells. The discretization of the city into hexagonal cells may introduce errors for short trips.

6 Conclusion

This paper studied the taxi dispatch problem in ride-hailing systems. The goal was to serve as many ride requests as possible while minimizing the cost of relocating idle vehicles. The problem was transformed into a Minimum Cost Maximum Flow problem on a directed graph. Hexagonal cells became nodes, drivers and requests became capacities, and vehicle movements became edges with repositioning costs.

The approach has two key theoretical properties. First, because the network has integer capacities, an optimal flow can be chosen to be integer-valued, so no rounding is required. Second, the time-greedy property guarantees that the number of served requests at the current time step is maximized even when future demand is uncertain, providing a worst-case performance guarantee.

The method achieves 97.16% of the performance of an oracle with perfect future knowledge. A linear programming version runs approximately 880 times faster than the standard MCMF algorithm in the reported experiments, enabling real-time decisions.

The key lesson is that network flow is a powerful tool for resource-allocation problems where discrete resources must be matched to demands over time and space. The same framework can also be applied to problems such as bike-sharing, ambulance dispatching, and logistics.

References

- [1] B. Kim, J. Kim, S. Huh, S. You, and I. Yang, “Multi-objective predictive taxi dispatch via network flow optimization,” *IEEE Access*, vol. 8, pp. 21437–21452, 2020.
- [2] F. Miao, S. Han, S. Lin, J. A. Stankovic, D. Zhang, S. Munir, H. Huang, T. He, and G. J. Pappas, “Taxi dispatch with real-time sensing data in metropolitan areas: A receding horizon control approach,” *IEEE Transactions on Automation Science and Engineering*, vol. 13, no. 2, pp. 463–478, Apr. 2016.
- [3] L. R. Ford, Jr. and D. R. Fulkerson, “Maximal flow through a network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956.
- [4] J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” *Journal of the ACM*, vol. 19, no. 2, pp. 248–264, Apr. 1972.
- [5] R. G. Busacker and P. J. Gowen, “A procedure for determining a family of minimum-cost network flow patterns,” Research Analysis Corporation, McLean, VA, USA, Tech. Rep. ORO-TP-15, 1960.
- [6] J. Kleinberg and É. Tardos, *Algorithm Design*. Boston, MA, USA: Pearson Addison-Wesley, 2005.